UNIVERSAL CODE SCANNER

17 · Resumen ejecutivo

Documentación de sistema

Versión analizada: **1.1.0+2**
Commit: **c1f9878**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. En una frase
2. El producto
3. La propuesta de valor, en una comparación
4. Qué se verificó ejecutando
5. Estado de salud: cinco señales
6. Lo que hay que saber antes de decidir
7. Riesgos, resumidos
8. Recomendaciones, con esfuerzo cuantificado
9. Qué está fuera del alcance de esta versión
10. Conclusión

Documento de decisión. Sin detalle técnico innecesario y con el esfuerzo de cada recomendación cuantificado.

En una frase

Universal Code Scanner es una **aplicación móvil de lectura de códigos QR y de barras que interpreta el contenido y evalúa su riesgo antes de permitir cualquier acción**, funcionando íntegramente en el dispositivo, sin servidores, sin cuentas y sin telemetría.

El producto

Aspecto	Estado
Versión analizada	`1.1.0+2`, commit `c1f9878`, rama `main`
Licencia	MIT
Tecnología	Flutter 3.44.7 · Dart 3.12.2
Tamaño del código propio	6 821 líneas de Dart en 59 archivos
Dependencias de terceros	19 paquetes directos, con versión exacta y lockfile versionado
Plataformas verificadas	Android (compilado y ejecutado en emulador) y web/PWA (compilada y publicada)
Plataformas que compilan sin ejecución verificada	iOS y macOS, en la CI
Plataformas sin motor nativo	Windows y Linux, cubiertas por la PWA
Distribución actual	APK en GitHub Releases y PWA en GitHub Pages
Estado de tiendas	Fuera del alcance de 1.1.0

La propuesta de valor, en una comparación

Pregunta	Un lector de códigos habitual	Universal Code Scanner
¿Qué hace al leer un enlace?	Lo abre	Muestra dominio, protocolo, ruta y consulta, y **espera confirmación**
¿Avisa si el enlace parece falso?	No	Sí: 16 señales locales, incluidas Punycode, alfabetos mezclados y credenciales embebidas
¿Dónde se procesa?	Con frecuencia, en un servidor	**Siempre en el dispositivo**
¿Qué sabe el fabricante de lo que escaneas?	Depende	**Nada**: no hay red saliente propia ni permiso de internet
¿Se guarda el historial?	A veces, en claro	Sí, **cifrado con AES-256-GCM**
¿Guarda contraseñas Wi-Fi y códigos de verificación?	Sí, habitualmente	**No** desde la pestaña Escanear

¿Sirve para inventariar?	Rara vez	Sí: sesiones de conteo con exportación a Excel
¿Sirve para generar códigos?	A veces	Sí: 10 simbologías a PNG y SVG

Qué se verificó ejecutando

Sobre el commit analizado, con el toolchain fijado por el propio proyecto:

Comprobación	Resultado
Resolución de dependencias	Correcta, sin cambios en el lockfile
Análisis estático estricto (`--fatal-infos`)	**0 hallazgos**
Suite de pruebas	**57 de 57 en verde**, en 24 segundos
Validación estructural offline	Sin errores

Estos cuatro resultados coinciden exactamente con lo que el repositorio declara en `VALIDATION.md` y en las insignias del `README.md`. **La documentación previa del proyecto es fiable**: no se encontró ninguna cifra inflada.

Estado de salud: cinco señales

Señal	Valoración	Base
Calidad del código	Buena	0 hallazgos con el nivel más estricto del analizador; 4 reglas adicionales sobre `flutter_lints`
Corrección criptográfica	Muy buena	AES-256-GCM autenticado, sin criptografía escrita a mano, con las tres propiedades críticas cubiertas por prueba
Cobertura de pruebas	Desigual	El 100 % de la criptografía tiene prueba; el 0 % de los almacenes de estado y de las pantallas grandes
Honestidad de la documentación	Muy buena	Declara sin adornos lo que no se ha verificado, incluidos los dispositivos físicos
Automatización	Buena	CI para cuatro plataformas, SBOM, inventario de licencias, checksums, Dependabot y un validador propio

Lo que hay que saber antes de decidir

Tres fortalezas que sostienen el producto

> **La promesa central está implementada de verdad.** `launchUrl` —la única forma de abrir algo— aparece **una sola vez** en las 6 821 líneas del proyecto, y está detrás de un diálogo. No es una afirmación de marketing.

La criptografía está bien hecha y bien probada. Tres propiedades difíciles y valiosas están implementadas y verificadas: una llave ausente nunca se regenera en silencio, una rotación fallida no deja material huérfano, y la migración verifica antes de borrar. Cualquiera de las tres, mal resuelta, produce pérdida irreversible de datos del usuario.

El proyecto declara sus límites. `VALIDATION.md` e `IMPLEMENTATION_STATUS.md` enumeran lo no verificado con el mismo detalle que lo verificado. Eso es infrecuente y reduce el riesgo de sorpresas.

Tres carencias que conviene atender

Ocho escenarios de regresión preparados y nunca ejecutados. `test_assets/` contiene imágenes con daño parcial, poca luz, deformación y varios códigos por imagen, cada una con su resultado esperado. El validador comprueba que los archivos existen; **ninguna prueba los decodifica**. Es una capacidad de detección que se cree tener y no se tiene.

Una garantía pública que el inventario no cumple. La documentación afirma que las contraseñas Wi-Fi y los códigos de verificación no se guardan automáticamente. Es cierto en la pestaña Escanear y **falso en la de Inventario**, que no aplica el filtro.

Nueve interruptores de capacidad que nadie consulta. `FeatureFlags` se presenta en tres documentos como una frontera de extensión activa. Se persiste y se restaura correctamente, y **ningún código de la aplicación la lee**.

Ninguna de las tres es un fallo de seguridad explotable. Las tres son divergencias entre lo que la documentación afirma y lo que el código hace.

Riesgos, resumidos

35 hallazgos registrados. Ninguno crítico: **no hay pérdida de datos, fuga de secretos ni vulnerabilidad explotable** en el estado actual.

Severidad	Cantidad	Qué implican
Alta	3	Divergencias entre documentación y código, y una capacidad de prueba no ejercida
Media	9	Deuda que crecerá: acciones sin fijar a SHA, gate local más laxo que la CI, exportación sin advertencia
Baja	23	Código muerto, inconsistencias menores, decisiones no documentadas

Detalle completo en 15-risks-and-technical-debt.md.

El riesgo que no está en esa lista

Ninguna versión se ha probado en dispositivos físicos de forma sistemática. Está declarado por el propio proyecto en tres documentos. La aplicación se instaló y arrancó en un emulador de Android API 36, y la matriz obligatoria de `../quality/DEVICE_TEST_MATRIX.md` sigue sin completar.

Es el mayor riesgo del producto y no es técnico: cámara real, biometría, PDF grandes, TalkBack y VoiceOver solo se pueden validar en hardware. Y la corrección principal de 1.1.0 —el fallo de lectura— **la reportó un teléfono real**

y solo un teléfono real puede cerrarla.

Recomendaciones, con esfuerzo cuantificado

Ordenadas por relación entre valor y coste. **Ninguna se aplicó en este trabajo**, que es exclusivamente documental.

#	Recomendación	Esfuerzo	Por qué
1	Completar la matriz de pruebas en dispositivos físicos	**2 a 3 jornadas** con 3 teléfonos	Es el único riesgo del producto que no se puede reducir desde el código
2	Decidir sobre el contenido sensible en el inventario: filtrarlo o documentar la excepción	**1 hora**	Hoy la documentación afirma algo que el código no cumple
3	Ejecutar el catálogo de regresión en una prueba de integración	**Media jornada**	El material ya está preparado y versionado
4	Probar la migración del historial desde preferencias	**3 horas**	Es el único flujo que puede perder datos del usuario
5	Aclarar en la documentación que las banderas de capacidad son una reserva, no un mecanismo activo	**30 minutos**	Elimina una expectativa falsa
6	Añadir `--fatal-infos` a la puerta de calidad local	**1 minuto**	Elimina la discrepancia entre local y CI
7	Fijar las 13 acciones de GitHub a SHA de commit	**30 minutos**	Cadena de suministro de los artefactos publicados
8	Añadir un diálogo antes de exportar historial o inventario	**2 horas**	El archivo sale en claro y la interfaz no lo advierte
9	Declarar un umbral mínimo de cobertura en la CI	**1 hora**	Hoy nada falla si la cobertura baja
10	Añadir un paso de CI que contraste el SBOM contra una base de CVE	**Media jornada**	El SBOM ya se genera; falta consultarlo

Total de las nueve recomendaciones técnicas (2 a 10): aproximadamente 2 jornadas. La primera —dispositivos físicos— es la más cara y la más valiosa.

Qué está fuera del alcance de esta versión

Declarado por el propio proyecto, no una carencia detectada:

- Firma de publicación, cuentas de desarrollador, fichas de tienda, capturas comerciales, AAB e IPA.
- Publicación en Google Play o App Store.
- Motor nativo para Windows y Linux.

- OCR, NFC, reputación remota de enlaces, consulta de productos, sincronización cifrada, API empresarial y modo kiosco.
- Interfaz en inglés: la infraestructura existe y el idioma **no se expone a propósito** hasta que todas las pantallas lean sus textos del sistema de traducción, para no entregar una interfaz a medio traducir.

Conclusión

Universal Code Scanner es un proyecto **maduro para su tamaño y honesto sobre sus límites**. Su promesa central —interpretar antes de actuar, sin enviar nada a ningún sitio— está implementada de forma verificable, y la parte más delicada del sistema, la criptografía, está bien construida y bien probada.

Las tres carencias de severidad alta son **divergencias entre lo que la documentación afirma y lo que el código hace**, no defectos de funcionamiento, y las tres se resuelven en menos de una jornada de trabajo entre las tres.

La decisión relevante no es técnica: **el producto no se ha probado en teléfonos reales de forma sistemática**, y esa es la brecha entre «compila, analiza y pasa sus pruebas» y «funciona en manos de una persona». Cerrarla cuesta dos o tres jornadas con tres dispositivos, y es el paso que separa esta versión de una candidata a publicación en tiendas.

Si necesitas…	Ve a
Entender qué hace, sin tecnicismos	01 · Descripción general, sección final
El detalle de los 35 hallazgos	15 · Riesgos y deuda técnica
La postura de seguridad completa	11 · Seguridad
El estado real de las pruebas	12 · Pruebas y calidad
Poner a alguien a trabajar en esto	18 · Guía para nuevos desarrolladores